

MODULE 07 · DOMAIN 3 - DEBUGGING AND PERFORMANCE

Moving Between Engines

It compiled. It ran. It returned different rows.

DOMAIN 3 of 5 MODULE 7 of 11 PROMPTS 3

Three kinds of migration problem

- ◆ **Does not compile.** Loud, immediate, and easy. You will fix these on day one.
- ◆ **Compiles, behaves differently.** Same function name, different rules.
- ◆ **Compiles, silently changes the result set.** Different rows. No error anywhere.
- ◆ Only the first kind gets caught by testing that the code runs.

THE POINT

Every migration horror story you have heard is the third kind. Nobody loses sleep over a syntax error.

First, the easy class. Find everything that will not survive.

What you actually type

Name both engines explicitly. Never say 'modern SQL'.

```
Source engine: <name and version>
```

```
Target engine: <name and version>
```

```
List every construct below that is not valid on the target.
```

```
For each one give:
```

- The line, quoted
- Why the target rejects it
- The target equivalent, if there is an exact one

```
If there is no exact equivalent, say so rather than offering  
the closest thing.
```

```
Do not rewrite the code. I want the inventory first.
```

THE LINE THAT MATTERS

Without the exact-equivalent rule, approximations arrive as translations.

Four that will not compile

Legacy procedure, moving to a Postgres-family warehouse.

```
01 ISNULL(mstart, '')          -- SQL Server. Use NVL or COALESCE
02 TRIM(a) + '|' + TRIM(b)     -- + is arithmetic here. Use ||
03 DECODE(status, 'A', 1, 0)   -- Oracle. Use CASE
04 WHERE ROWNUM <= 100         -- Oracle. Use LIMIT
05
06 -- All four fail loudly on the target. None is dangerous.
07 -- The second one is worth a second look, though.
08 -- If both operands are numeric, + is valid and silent.
```

THE ONE TO NOTICE

Three of these throw. The fourth throws only when the operands happen to be text.

Now the dangerous class. Same syntax,
different **meaning**.

Where engines quietly disagree

- ◆ **Empty string and null.** Oracle treats an empty string as null. Most others do not.
- ◆ **Concatenation with null.** Oracle ignores it. Postgres family returns null.
- ◆ **Integer division.** Some engines return a decimal. Others truncate.
- ◆ **Implicit casts, date arithmetic, string truncation.** All differ, all silently.

THE POINT

Every item here is valid syntax on both engines. Not one of them will raise anything.

Concatenation, meeting a null

A hash key built from three components.

```
01 f_hash(source_key || UPPER(part_no) || UPPER(site_code))
02
03 -- On Oracle, if site_code is null:
04 --   null is treated as an empty string
05 --   the hash is built from the other two components
06
07 -- On a Postgres-family engine, if site_code is null:
08 --   the whole expression evaluates to null
09 --   the hash is null, and every join on it fails
10
11 -- Same line. Same syntax. Rows silently stop matching.
```

WHAT ACTUALLY HAPPENS

The keys do not error. They come out null, the joins miss, and the rows land on a default value.

What you actually type

Ask about what stays the same, not what changes.

```
Source engine: <name>. Target engine: <name>.
```

```
Find every expression below that is VALID on both engines but  
may produce a different result. Ignore anything that fails to  
compile on the target; I have those already.
```

```
For each one:
```

- What each engine returns
- The data condition needed to trigger the difference
- A query telling me whether that condition exists

```
Focus on nulls, empty strings, division and implicit casts.
```

THE BULLET THAT SCALES IT

The third bullet turns theoretical risks into a short list of real ones.

What it returned, and what survived testing

- ◆ **Eleven expressions flagged** as potentially behaving differently.
- ◆ **Seven ruled out by query.** The triggering condition does not occur in the data.
- ◆ **Three confirmed live.** Null site codes exist. Hash keys would break.
- ◆ **One undecidable.** Depends on data that has not arrived yet.

THE RESULT

Eleven risks became three fixes and one thing to monitor. That is the point of asking for the queries.

Verification, when you cannot query both systems at once.

Why the previous approach fails

- ◆ The two result sets live in different databases. You cannot subtract one from the other.
- ◆ Exporting both and diffing works, until the volumes are real.
- ◆ **Compare aggregates instead.** Counts, sums, and grouped checksums.
- ◆ **Then compare rows for a sampled slice.** Aggregates find drift. Rows find the cause.

THE TWO LAYERS

Aggregates tell you that something differs. Only rows tell you what.

What you actually type

Two queries. One per engine. Same shape.

```
I need to verify a migrated query returns the same results.  
The two result sets are on different engines and cannot be  
joined. Write me a matched pair of verification queries.
```

```
Each must return the same shape, so I can diff the outputs:
```

- A grouping key at a grain coarse enough to be readable
- Row count per group
- Sum of each numeric measure per group
- Count of nulls per nullable column, per group

```
Write them in the correct dialect for each engine. Do not  
assume a function exists on both.
```

THE ROW THAT EARNS ITS PLACE

The null counts are what catch the concatenation problem. Sums alone would not.

Two kinds of verification

SAME ENGINE

Set difference

- Both directions, EXCEPT
- Definitive for the data tested
- One query
- Not available across engines

ACROSS ENGINES

Matched aggregates

- Two queries, diffed by eye
- Finds drift, not its cause
- Cheap at any volume
- Then sample rows where it differs

THE TIEBREAKER

Neither is a proof. Both are far better than reading the translated code and agreeing with it.

The dialect audit finds a function used forty times across the codebase. The model offers a target equivalent and notes it is 'functionally similar'.

You have three weeks and two hundred procedures to migrate.

What do you do with that phrase?

- ☐ A Accept it. Forty instances is too many to check individually
- ☐ B Ask specifically how it differs, before using it anywhere
- ☐ C Use it, and add it to a list to revisit if problems appear
- ☐ D Find a different equivalent that is exact

◆ PAUSE AND DECIDE

What do you do with that phrase?

A Accept it

*Forty instances is the argument for checking, not against.
One difference becomes forty defects.*

C Revisit if problems appear

*The problems this module is about do not appear. That is
their defining feature.*

B Ask how it differs

*One prompt, applied once, covers all forty. This is the
cheapest question in the module.*

D Find an exact one

*There may not be one. Assuming there is sends you looking
for something that does not exist.*

THE SIGNAL

Functionally similar is a phrase that means 'differs in ways I have not enumerated'.

The semantic drift prompt flags that null concatenation behaves differently. It gives you a query to check whether any hash key component is ever null.

You run it. Zero rows. No component is ever null in the current data.

Is this risk closed?

- ☐ A Yes. The condition does not occur, so the behaviour cannot differ
- ☐ B No. Add a null guard to every hash expression anyway
- ☐ C No. The source system may start allowing nulls later
- ☐ D Yes for now, and it needs a constraint or a monitor

◆ PAUSE AND DECIDE

Is this risk closed?

- A

 Closed
True today. It also means the safety of your hash keys now depends on an untracked assumption.
- B

 Guard everything
Defensible, and it changes the hash values. Every key you have already stored would move.
- C

 It might change later
Correct and unhelpful on its own. Every check in this module could be invalidated later.
- D

 Closed, with a guard on it
Records the assumption in a place that will notice when it stops being true.

THE SIGNAL

You have discovered a load-bearing assumption. Write it into a constraint, not into your memory.

The matched aggregate queries agree on row counts and on every measure sum. The null counts differ on one column, by 1,400 rows.

That column is not used in any measure. It feeds a hash key.

How serious is this?

- ☐ A Minor. The measures all agree, so the numbers are right
- ☐ B Serious. Those 1,400 rows will fail to join downstream
- ☐ C Unknown until you sample the rows and look
- ☐ D Cosmetic. Null counts often differ after a migration

◆ PAUSE AND DECIDE

How serious is this?

A Minor

The measures agree in this query. The hash feeds joins in other queries, where they will not.

C Sample and look

You should, to confirm. The mechanism is already identified, so this is verification not diagnosis.

B Serious, joins will fail

This is exactly the concatenation case, caught by the one check that sums would have missed.

D Cosmetic

Null counts differing is never cosmetic. It means two engines disagree about a value.

THE SIGNAL

This is the finding the null count row was added for. Sums and counts both passed it.

Migration is done. Everything verified. A colleague suggests running both systems in parallel for a month and comparing outputs daily.

It costs one engineer roughly two hours a week.

What is the strongest case for doing it?

- ☐ A It catches anything the verification queries missed
- ☐ B It exercises data conditions that did not exist during testing
- ☐ C It gives the business confidence in the migration
- ☐ D It provides a rollback path if something goes wrong

◆ PAUSE AND DECIDE

What is the strongest case for doing it?

- A

 Catches what was missed
True and vague. It does not say why a month of running would catch what a query did not.
- B

 New data conditions
Names the actual mechanism. Every check you ran was against data that existed at the time.
- C

 Business confidence
A real benefit and a weak argument. Confidence without a mechanism is just reassurance.
- D

 Rollback path
Parallel running is not what gives you rollback. Keeping the old system running does.

THE SIGNAL

The undecidable item from the drift audit is exactly the thing a month of parallel running resolves.

Take this to work tomorrow

- ◆ Name both engines and versions in every migration prompt. Never say 'standard SQL'.
- ◆ Run the drift audit separately from the compile audit. They find different things.
- ◆ For every drift finding, ask for the query that tells you whether it applies to your data.
- ◆ Include null counts per column in every verification. Sums will not catch a null key.

THE PAYOFF

Seven of eleven risks were eliminated by running a query. That ratio is typical.

The dangerous code is the code that needed **no** translation.